UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Universidad decana de América

Facultad de Ingeniería de Sistemas e Informática

Escuela Profesional de Ingenieria de Sistemas.

Estructura de Datos

Prof. Gilberto A. Salinas

Semana 02
Métodos de Búsqueda

Ciclo 2026-1

Agenda

1. Introducción
2. Definición
   - Características
   - Ventajas/Desventajas
3. Operaciones básicas
   - Análisis, diseño e implementación
4. Resumen
5. Tablas Hash
6. Referencias

1. **Introducción formal**

Busqueda del elemento vx en x = {e1,e2, …, en}

ENTRADA

- Secuencia de n elementos: {v1, v2, …, vN}
- Un elemento : vx

SALIDA:

- Un entero i, tal que vx = vi
- 0 si vx != vi, para i = 1,2,…, n

Ejemplo:

ENTRADA: { -5, 5, 7, 10, 14} y vx = 13

SALIDA: vx: “No hallado”

2. Definición

- La búsqueda de un elemento “dato” en un conjunto de elementos consiste en:
  - Determinar si “dato” pertenece al conjunto y la posición que ocupa en el.
  - Determinar si “dato” no pertenece al conjunto de elementos.

Diseno y especificacion de la operación. Sea x un vector de tipo TD

ACCION buscarDato(Vector x, ENTERO dx, TD dato)

//Busca el elemento dato en el vector x

PRE $x = \{\} \vee x = \{e1,e2, \ldots, en\} \wedge dato \in x$

POS: “Vector vaciooo…” $\vee$ (dato, “No se encuenrta…” $\vee$ dato, “Se encuentra en la posición ”, i)

1. **Algoritmo de busqueda**

Definicion.

Son algoritmos para encontrar un dato dentro de una estructura o arreglo

- Se han desarrollado un conjunto de algoritmos de busqueda que varian en eficiencia, complejidad y cantidad de elementos del dominio de busqueda
- Es posible elegir el algorimo mas adecuado si se conoce el orden en que se encuentran los elementos.

1. **Definicion**

Los algoritmos de busqueda no es solo para localizar valores textuales o numericos. Desempenan un papel crucial en:

- Analisis de datos
- Gestion de base datos
- Inteligencia artificial.
- Optimizacion de motores de busqueda
- Problemas de optimizacion.
- Procesamiento d imagenes
- Determinar rutas en robotica

1. Introducción

1. **Introducción**

   Los algoritmos de busqueda son fundamentales en la informatica, disenados para localizar un elemento epexcifico dentro de una estructura de datos, estos varian en funcion del tipo de estructura y de la naturaleza de los datos.

2. **Tipos de algoritmo de busqueda**
   - Busqueda lineal (secuencial)
   - Busqueda binaria
   - Busqueda por salto
   - Busqueda por interpolacion
   - Busqueda exponencial
   - Busqueda fibonacci
   - Busqueda hash

1. **Busqueda lineal o secuencial**

1. **Descripcion**

   Consiste en recorrer la estrucutra de datos elemento a elemento hasta ubicar el datos deseado.

   - **Complejidad temporal**. En el mejor caso O(1), peor caso O(n).

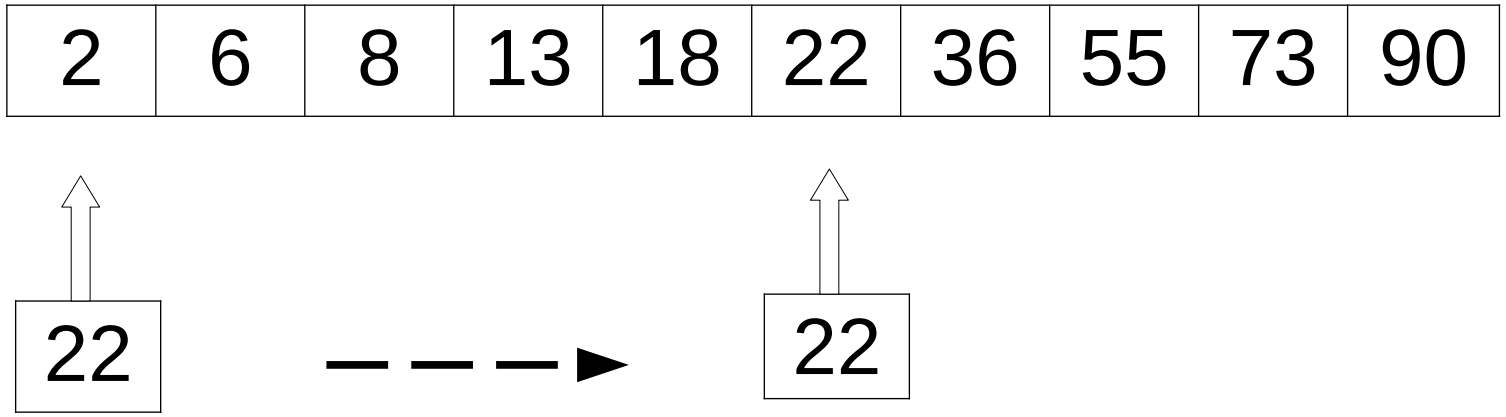
   Pros

   - Sencillo y facil de implementar
   - Funciona en arreglos con los elementos desordenados
   - Adecuado para conjunto de datos pequenos

   Contras

   - No es adecuado para grandes conjuntos de datos

1. **Busqueda lineal o secuencial**



3. Operaciones

- **Búsqueda secuencial.** Sea x un vector de enteros

ACCION busquedaSecuencial(Vector x, ENTERO dx, TD dato)

//Busca el elemento dato en el vector x

PRE x = {} $\vee$ x = {e1,e2, …, en} $\wedge$ dato $\in$ x

POS: “Vector vaciooo…” $\vee$ (dato, “No se encuenrta…” $\vee$ dato, “Se encuentra en la posición ”, i)

3. Operaciones

```
ACCION  búsquedaSecuencial (Vector x, ENTERO dx, TD dato)
    ENTERO i ← 0, val
    BOOL hallado ← falso
    SI(dx>0)
        MIENTRAS( i<dx Y NO hallado)
            Val ← x[i]
            SI(val = dato)
                ESCRIBIR(dato, “se encuentra en la posicon”, i+1)
                hallado ← verdad
            FIN_SI
            i ← i +1
        FIN_MIENTRAS
        SI(NO hallado)
            ESCRIBIR(dato, “no se encuentraaaa…..”
        FIN_SI
    SINO
        ESCRIBIR(“Vector vaciooo…”)
    FIN_SI
FIN_ACCION
```

2. **Busqueda Binaria**

1. **Descripcion**

   Consiste en dividir repetidamente la lista ordenada por la mitad hasta encontrar el dato deseado (objetivo es el valor central) o hasta determinar que el valor no esta en la lista.

   - **Complejidad temporal**. En el mejor caso O(1), peor caso O(log n).

   Pros

   - Algoritmo eficaz para grandes colecciones de datos ordenados
   - Algorittmo mucho mas eficiente que la busqueda lineal
   - Implementacion sencilla

   Contras

   - Se necesita que los elementos esten ordenados
   - Requiere acceso aleatorio al arreglo, lo que puede ser una desventaja en algunas estructurasm (listas enlazadas)

1. **Busqueda binaria**

36

0	1	2	3	4	5	6	7	8	9
2	6	8	13	18	22	36	55	73	90

36

0	1	2	3	4	5	6	7	8	9
2	6	8	13	18	22	36	55	73	90

36

0	1	2	3	4	5	6	7	8	9
2	6	8	13	18	22	36	55	73	90

36

0	1	2	3	4	5	6	7	8	9
2	6	8	13	18	22	36	55	73	90

3. Operaciones

- **Búsqueda binaria.** Sea x un vector de enteros cuyos elementos estan ordenados.

ACCION busquedaBinaria(Vector x, ENTERO dx, TD dato)

//Busca el elemento dato en el vector x

PRE x = {} $\vee$ x = {e1>e2> … >en} $\wedge$ dato $\in$ x

POS: “Vector vaciooo…” $\vee$ (dato, “No se encuenrta…” $\vee$ dato, “Se encuentra en la posición ”, i)

3. Operaciones

```
ACCION  búsquedaBinaria (Vector x, ENTERO dx, TD dato)
    ENTERO i ← 0, ini, fin,med,val
    BOOL hallado ← falso
    Ini ← 1,  fin ← dx
    med ← (ini+fin)/2
    SI(dx>0)
        MIENTRAS( ini<fin Y x[med]  ≠ dato)
            SI(dato)< x[med]
                fin ← med – 1
            SINO
                ini ← med + 1
            FIN_SI
            med ← (ini + fin)/2
        FIN_MIENTRAS
        SI(dato = x[med])
           ESCRIBIR(dato, "encontrado  en la posición: ",med)
         SINO
           ESCRIBIR(dato, " no encontradoooo …: ")
        FIN_SI
    SINO
        ESCRIBIR("Vector vaciooo…")
    FIN_SI
FIN_ACCION
```

3. **Busqueda por salto o bloques**

1. **Descripcion**

   En lugar de buscar cada elemento, este metodo busca en intervalos fijos o “saltos”. Hace una busqueda lineal (reutiliza) en esa region si encuentra un intervalo en el que podria estar el valor deseaado.

   - **Complejidad temporal**. $O(\sqrt{n})$.

Pros

- Es mejor que la busqueda lineal para arreglos grandes y uniformemente distribuidos.
- La implementacion es menos complicada que los algorimos de busqueda binaria o ternaria

Contras

- Se necesita que los elementos esten ordenados
- Disminucion de la eficacia si el arreglo no esta distribuida uniformemente.

4. **Busqueda por bloques**

0	1	2	3	4	5	6	7	8	9
1	2	3	4	5	6	7	8	9	10

5 no esta en [1]

0	1	2	3	4	5	6	7	8	9
1	2	3	4	5	6	7	8	9	10

5 no esta en [2, 3]

0	1	2	3	4	5	6	7	8	9
1	2	3	4	5	6	7	8	9	10

5 esta en [4, 5, 6, 7]

4. **Busqueda por interpolacion**

1. **Descripcion (W. Peterson 1957)**

   En realidad es una mejora de busqueda binaria para valores uniformemente distribuidos. Estima una posicion del valor deseado y puede localizarlo mas rapido que en la busqueda binaria si la estimacion es cercana. (como buscar la pagina de un libro, diccionario o guia telefonica)

   - **Complejidad temporal**. En el mejor caso O(1) y el [peor caso O(n).

   Pros

   - Algoritmo eficaz para grandes colecciones de datos ordenados
   - Algorittmo mucho mas eficiente que la busqueda lineal
   - Puede identificar si el elemento objetivo no esta en el conjunto’

   Contras

   - Se necesita que los elementos esten ordenados
   - Disminucion de la eficacia si el arreglo no esta distribuida uniformemente.

4. **Busqueda por interpolacion**

x

5

0	1	2	3	4	5	6	7	8	9
1	2	3	4	5	6	7	8	9	10

El algoritmo calcula la posicion:

Pos = bajo + ((dato – x[bajo])(alto – bajo))/(x[alto]) – x[bajo]

Donde:
Bajo: Inidce mas bajo del rango actual
Alto: Inidce mas alto del rango actual

x

5

0	1	2	3	4	5	6	7	8	9
1	2	3	4	5	6	7	8	9	10

√

5

0	1	2	3	4	5	6	7	8	9
1	2	3	4	5	6	7	8	9	10

5. **Busqueda exponencial**

1. **Descripcion**

   Esta busqueda divide la estructura o arreglo (rangos) utilizando portencias exponenciales de 2 y se encuentra el valor deseado aplicando busqueda binaria.

   - **Complejidad temporal.** O(log n). O(log i) donde es la posicion del dato

Pros

- En algunos casos esta busqueda es mejor que la busqueda binaria.
- Puede manejar espacios de busqueda complejos, como listas ilimitadas o infinitas
- Es mas favorable si el elemento se encuenbtra cercana al inicio del arreglo.

Contras

- Se necesita que los elementos esten ordenados
- El algoritmo es ineficiente para arreglos pequenos.

https://en-m-wikipedia-org.translate.goog/wiki/Exponential_search?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=wa

4. **Busqueda exponencial**

Tb= pow(2, k)

Dato: 5

Como Funciona:

- **Determinacion del rango.**
  Se inicio con indice 1. y se va duplicando el valor: (1, 2, 4, 8,)

- **Busqueda binaria.**
  Una vez identificado el rango entre el ultimo y el anterior, se realiza la busqueda binaria.

0	1	2	3	4	5	6	7	8	9
1	2	3	4	5	6	7	8	9	10

0	1	2	3	4	5	6	7	8	9
1	2	3	4	5	6	7	8	9	10

0	1	2	3	4	5	6	7	8	9
1	2	3	4	5	6	7	8	9	10

6. **Busqueda Fibonacci**

1. **Descripcion**

   En esta busqueda se utiliza la serie de numeros Fibonacci para dividir los datos en bloques de diferentes tamanos y luego se localiza el valor deseado de forma similar a la busqueda binaria. F = {0,1,1,2,3,5,8,}

   - **Complejidad temporal**. O(logn).

   Pros

   - Uno de los metodos de busqueda mas eficiente.
   - Grandes conjunos de datos y no requier espacio adicional

   Contras

   - Se necesita que los elementos esten ordenados

6. **Busqueda Fibonacci**

1. **Descripcion**

   Buscamos el Fibonacci que sea >= n numero de elementos de array. F_k = 13, F_{k-1} = 8, F_{k-2} = 5

   - **Complejidad temporal**. O(logn).

   Pros

   - Uno de los metodos de busqueda mas eficiente.
   - Grandes conjunos de datos y no requier espacio adicional

   Contras

   - Se necesita que los elementos esten ordenados

7. **Busqueda Hash**

1. **Descripcion**

   Mediante la funcion hash, h(dato), los datos se convierten en un indice de la estructura donde pueden almacenarse o recuperarse.

   - **Complejidad temporal**. O(1), tanto para la insercion como para la busqueda, puede variar en funcion de la eficiencia de la funcion hash y el manejo de las colisiones.

   Pros

   - Busqueda y almacenamiento oficiente
   - Una ocupacion menor al 75% es buena.

   Contras

   - Operacion costosa si hay necesidad de ampliar la tabla.
   - Desaprovecxhamiento de memoria si se reserv espacio para todos los lementos, se consume mas meoria de la necesaria

4. Resumen

- Siempre vamos a tener necesidad de hacer búsqueda de la información
- Debemos utilizar el método de búsqueda adecuado

Semana 02
Tablas hash

Ciclo 2026-1

Agenda

1. Introducción
2. Función hash
3. Generadores de función hash
4. Resolución de colisiones
5. Hashing abierto
6. Referencias

1. Introducción

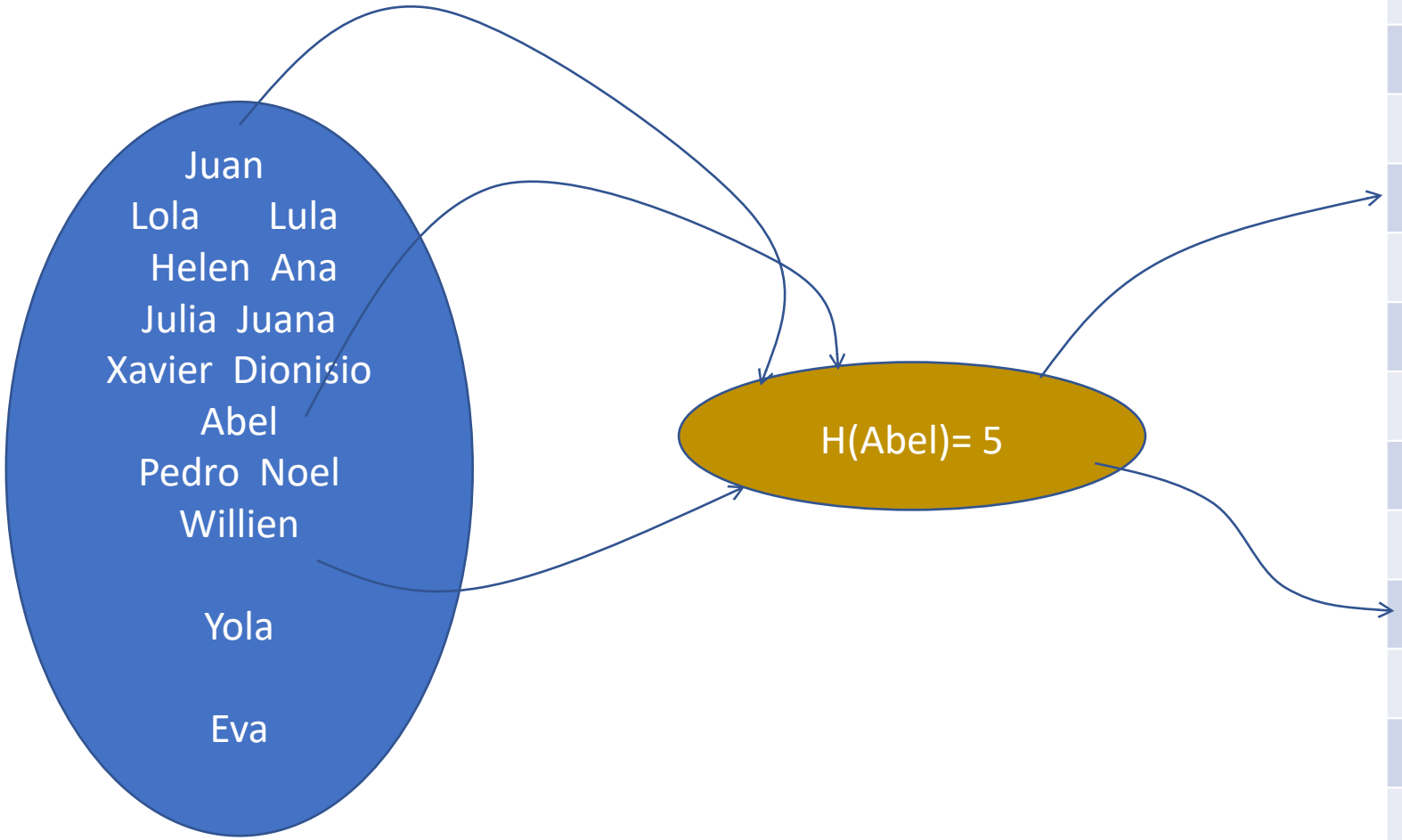
1. Una búsqueda diferente a los anteriores que consiste en comparación entre valores clave, sino encontrando alguna ***función h(k)*** que nos de directamente la dirección de la clave k en la tabla.
2. Existen algoritmos para construir funciones Hash que son utilizadas para organizar palabras clave en un compilador de forma que la búsqueda de cualquiera de las palabras clave se realice en tiempo constante.
3. Las funciones que evitan valores duplicados son sorprendentemente difíciles de encontrar, incluso para tablas pequeñas.
4. El objetivo será pues encontrar una función hash que provoque el menor número posible de colisiones (ocurrencias de sinónimos), aunque esto es solo un aspecto del problema, el otro será el de diseñar métodos de resolución de colisiones cuando éstas se produzcan.

2. Funcion Hash

1. El primer problema que debemos abordar es el cálculo de la función hash que transforma claves en localizaciones en la tabla.
2. Más concretamente, necesitamos una función que transforme claves(normalmente enteros o cadenas de caracteres) en enteros en un rango [0..M-1],
   - Donde M es el número de registros que podemos manejar con la memoria que disponemos como factores a tener en cuenta.
   - Para la elección de la función *h(k), debe ser una* que minimice las colisiones y que sea relativamente rápida y fácil de calcular, aunque la situación ideal sería encontrar una función *h* que generara valores aleatorios uniformemente sobre el intervalo [0..M-1]

2. Funcion Hash

1. Hash



dir	Info
1	
2	
3	
4	
5	Abel
6	
7	
8	
9	
10	
11	Willien
12	
13	
14	

3. Generadores de funcione Hash

1. **Hash multiplicativo.** Esta técnica trabaja multiplicando la clave *k* por sí misma o por una constante, usando después alguna porción de los bits del producto como una localización de la tabla hash
   - Cuando la elección es multiplicar *k* por sí misma y quedarse con alguno de los bits centrales, el método se denomina el cuadrado medio. Ejemplo:
   - k*k = abcd...lmn...xyz
     - h(k) = lmn $\vee$
     - h(k) = bcd $\vee$
     - h(k) = xyz
2. **Hashing por división.** La función se calcula simplemente como **h(k) = k mod M** usando el 0 como el primer índice de la tabla hash de tamaño M.

4. Resolución de colisiones

1. Estudiaremos tres métodos básicos de resolución de colisiones,
   - Uno de ellos depende de la idea de mantener listas enlazadas de sinónimos
   - Los otros dos, del cálculo de una secuencia de localizaciones en la tabla hash hasta que se encuentre una vacía.
   - El análisis comparativo de los métodos se hará en base al estudio del número de localizaciones que han de examinarse hasta determinar donde situar cada nueva clave en la tabla.

4. Resolución de colisiones

1. Para todos los ejemplos el tamaño de la tabla será M=13 y la función hash $h_1(k)$ que utilizaremos será:
   - **HASH = Clave Mod M**
   - Los valores de la clave k que consideraremos son los expuestos en la tabla:.

j	h_j	$h_i(h_j)$
1	119	2
2	85	7
3	43	4
4	141	11
5	72	7
6	91	0

5. Hashing abierto

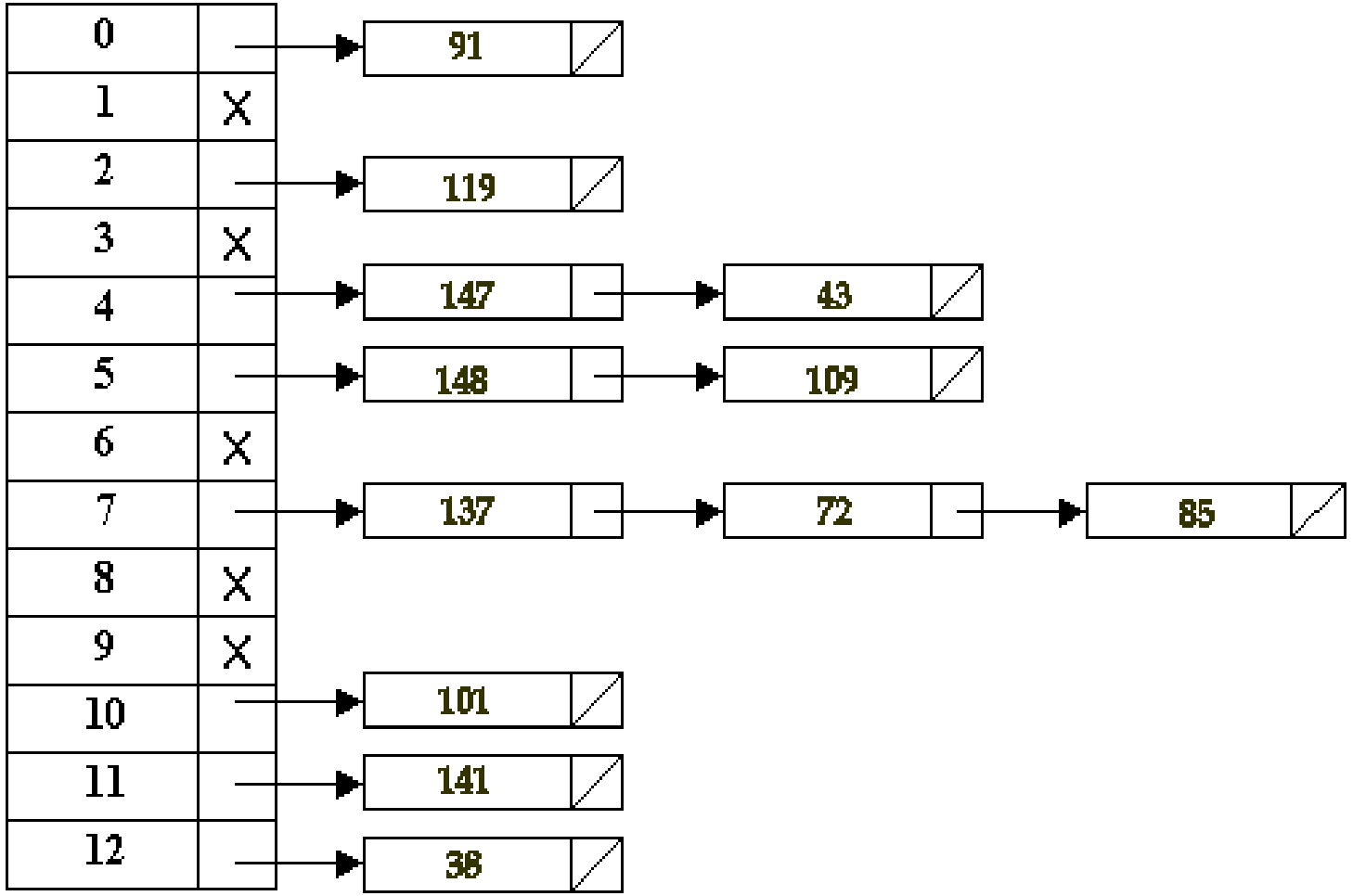
1. Este método se conoce normalmente con el nombre de *encadenamiento separado* y obviamente la cantidad de tiempo requerido para una búsqueda dependerá de la longitud de las listas y de las posiciones relativas de las claves en ellas.
2. La manera más simple de resolver una colisión es construir, para cada localización de la tabla, una lista enlazada de registros cuyas claves caigan en esa dirección.

5. Hashing abierto

1. Las listas se mantienen en orden esto puede verse como una generalización del método de búsqueda secuencial en listas.
2. La diferencia es que en lugar de mantener una sola lista con un solo nodo cabecera se mantienen M listas con M nodos cabecera de forma que se reduce el número de comparaciones de la búsqueda secuencial en un factor de M (en media) usando espacio extra para M punteros. Para nuestro ejemplo y con la alternativa LIFO, la tabla quedaría como se muestra en la siguiente figura:

5. Hashing abierto

1. Hashing abierto.



Referencias

- Joyanes L. y Zahonero I. (2004) Algoritmos y estructura de datos. Una perspectiva en C. Madrid España: McGraw-Hill.
- Cortez A. (2013) Algoritmica técnicas algorítmicas. Lima. Peru: CEPREDIN
- Joyanes L. y Zahonero I. (2008) Estructura de datos en Java. Madrid, España: McGraw-Hill.
- Cairo O y Guardati S. (2006) Estructura de datos. Mexico: McHraw-Hill.
- Villalobos J. (1996) Diseño y Manejo de Estructuras de Datos en C. Santa Fe de Bogota, Colombia: McGraw-Hill.
- Ficheros en C y C++ .[Ultima actualización 13.06.2017] [Accesado en Lima, 19.05.2020, 1820 horas], http://c.conclase.net/ficheros/
- http://decsai.ugr.es/~jfv/ed1/tedi/cdrom/docs/tablash.html

Referencias

- Joyanes L. y Zahonero I. (2004) Algoritmos y estructura de datos. Una perspectiva en C. Madrid España: McGraw-Hill.
- Cortez A. (2013) Algoritmica técnicas algorítmicas. Lima. Peru: CEPREDIN
- Joyanes L. y Zahonero I. (2008) Estructura de datos en Java. Madrid, España: McGraw-Hill.
- Cairo O y Guardati S. (2006) Estructura de datos. Mexico: McHraw-Hill.
- Villalobos J. (1996) Diseño y Manejo de Estructuras de Datos en C. Santa Fe de Bogota, Colombia: McGraw-Hill.
- Algoritmo de búsqueda .[Ultima actualización 12.05.2020] [Accesado en Lima, 18.05.2020, 1820 horas], https://es.wikipedia.org/wiki/Algoritmo_de_b%C3%BAsqueda
- Algoritmo de Ordenamiento.[Ultima actualización 28.05.2020] [Accesado en Lima, 29.05.2020, 1420 horas], https://es.wikipedia.org/wiki/Algoritmo_de_ordenamiento